

# **Baseline Machine Learning from Scratch**

*Task 10 Theory Notes — Regression, Classification, and Clustering on AirQualityUCI*

Name: Samaira Singh

Roll Number: 251090051002

Repository: [github.com/samaira-s/Synergy\\_TP](https://github.com/samaira-s/Synergy_TP)

Dataset: AirQualityUCI (UCI Machine Learning Repository)

## 1. Machine Learning Problem Types

Supervised learning uses data where every training row has a known correct answer (a label or target) attached, and the goal is to learn a mapping from inputs to that answer. Unsupervised learning has no such answer attached — the algorithm only sees input features and must find structure on its own. Regression and classification are both supervised: regression predicts a continuous numeric value (e.g. this task's CO(GT) concentration, in mg/m<sup>3</sup>), while classification predicts a category from a fixed set (e.g. this task's derived pollution\_class: high vs low). Clustering is unsupervised: it groups rows by feature similarity (here, temperature and humidity) without ever being told what the "correct" grouping is.

## 2. Dataset Framing

A sample is one row of data — in this dataset, one hour's worth of sensor and weather readings. A feature is one input column used to make a prediction (e.g. PT08.S1(CO), T, RH). A label or target is the value being predicted (CO(GT) for regression; the derived pollution\_class for classification). The feature matrix X is every feature column stacked together as a 2D array (rows = samples, columns = features); the target vector y is just the target column, as a single 1D array in the same row order as X. Every model in this task is trained by learning a relationship between X and y.

## 3. Regression

The regression task predicts CO(GT), a continuous, real-valued measurement from a reference analyzer, using the PT08 low-cost sensor readings plus temperature, relative humidity, and absolute humidity as features. A residual is the gap between an actual and predicted value for one row (actual - predicted); loss is a single number summarizing residuals across the whole dataset, typically the average of squared residuals (MSE). The baseline regression predictor used here always predicts the training set's mean CO(GT) value, regardless of input — a trained model is only useful if it clearly beats this. Linear regression intuition:  $\text{fit predicted} = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_k \cdot x_k + b$ , a weighted sum of features plus an offset, choosing weights and bias that minimize the loss.

## 4. Classification

A class label is the category being predicted; this task uses binary classification (exactly two classes: 0 = low pollution, 1 = high pollution), created by splitting CO(GT) at the training set's median value. Logistic regression intuition: despite the name, it is used for classification — it computes the same weighted sum as linear regression, then passes that sum through a sigmoid function ( $1 / (1 + e^{-z})$ ) to squash it into a valid probability between 0 and 1. A decision threshold (default 0.5) then converts that probability into an actual 0/1 prediction: probability  $\geq$  threshold predicts class 1. The majority-class baseline always predicts whichever class was more frequent in the training set, without looking at the input at all.

## 5. Clustering

Clustering groups rows by feature similarity without any label — KMeans intuition: choose a number of clusters k, place k centroids (initially at k randomly chosen data points), then repeat two steps until stable: (1) assign every row to its nearest centroid by distance, (2) move each centroid to the mean position of the rows currently assigned to it. A centroid is the average feature values of a cluster's current members. Cluster assignment is which centroid (0..k-1) a row currently belongs to. Cluster interpretation is a manual, post-hoc step: the algorithm does not know what a cluster "means" — in this task, clusters were interpreted afterward by

examining their centroid temperature/humidity values (e.g. a hot-dry cluster vs a cool-humid cluster).

## 6. Gradient Descent

Parameters are the values being learned — weights (one per feature) and a bias term. The loss function is the number gradient descent is trying to minimize (MSE for linear regression; log loss for logistic regression). The gradient, for each parameter, indicates the direction of steepest loss increase; since we want to decrease loss, parameters are updated in the opposite direction of the gradient. The learning rate controls how large each update step is. One iteration is one full pass of computing the gradient and updating parameters; convergence is reached when the loss stops meaningfully decreasing between iterations. Feature scale matters because gradient descent's step size is effectively different per feature when features are on very different numeric scales (e.g. PT08 sensor values in the hundreds/thousands vs humidity as a fraction) — this can make training slow or unstable, which is why all features in this task were standardized (mean 0, standard deviation 1) using statistics computed only from the training set before training any model.

## 7. Train, Validation, and Test

A model must be evaluated on data it never saw during training, because a model can fit its training data very closely without that meaning it has learned anything generalizable — evaluating on the same data used to train would overstate how good the model actually is. This task uses a chronological (not random) 70/15/15 train/validation/test split, since the AirQualityUCI data is an hourly time series and adjacent hours are highly correlated; a random shuffle-based split would let near-duplicate hours leak between train and test, artificially inflating apparent performance. The test set should not be used repeatedly during model selection or tuning, because doing so effectively lets the model (or the person tuning it) indirectly "see" and adapt to the test set over many attempts, defeating its purpose as a genuinely unseen check — repeated tuning against a fixed test set should use the validation set instead, reserving the test set for a final, one-time evaluation.

## 8. Regression Evaluation Metrics

MAE (mean absolute error) is the average of  $|\text{actual} - \text{predicted}|$  across all rows — directly interpretable in the target's original units. MSE (mean squared error) is the average of the squared residuals, which penalizes large errors more heavily than small ones due to squaring. RMSE is the square root of MSE, bringing the error metric back into the target's original units while still weighting large errors more than MAE does. R-squared measures the fraction of variance in the target explained by the model, relative to always predicting the mean:  $R^2 = 1 - (\text{sum of squared residuals} / \text{total variance in } y)$ .  $R^2 = 1$  is a perfect fit;  $R^2$  near 0 (or negative) means the model performs no better than (or worse than) always guessing the mean.

## 9. Classification Evaluation Metrics

Accuracy is the fraction of all predictions that were correct. Precision is  $TP / (TP + FP)$ : of everything predicted as the positive class, how much actually was. Recall is  $TP / (TP + FN)$ : of everything that actually was the positive class, how much was correctly caught. F1-score is the harmonic mean of precision and recall, penalizing a large gap between the two more than a simple average would. A confusion matrix shows all four outcome counts at once — true positive, true negative, false positive, false negative — making it possible to see exactly what kind of mistakes a model is making, not just how many.

## 10. Clustering Evaluation Metrics

Inertia is the sum of squared distances from every point to its own assigned centroid, summed across all clusters — lower inertia means tighter, more compact clusters, but inertia alone cannot indicate whether the chosen number of clusters,  $k$ , is actually appropriate (it always decreases as  $k$  increases). Silhouette score measures, for each point, how much closer it is to its own cluster than to the nearest other cluster, averaged across all points; it ranges roughly from -1 (likely wrong cluster) to +1 (well-separated, well-matched cluster), and unlike inertia, is comparable across different values of  $k$ . Cluster counts (how many points fall into each cluster) reveal whether clusters are reasonably balanced or whether one cluster has absorbed almost everything. Visual inspection (plotting the clusters in feature space) remains necessary because none of the numeric metrics alone confirm that a clustering is physically or practically meaningful.

## 11. Baseline Predictors

Every trained model in this task is compared against a deliberately "dumb" baseline: mean-prediction for regression, majority-class for classification. This comparison exists because a trained model's raw metric score means little in isolation — a model achieving 90% accuracy sounds strong until it's revealed the majority class baseline achieves 85% accuracy for free, without learning anything from the input features at all. A model is only demonstrably useful if it clearly and meaningfully outperforms this trivial reference point.

## 12. Data Leakage

Target leakage occurs when a feature is (directly or indirectly) derived from or highly dependent on the target itself, giving the model an unfair, unrealistic shortcut — e.g. including a column already correlated with the classification label by construction. Preprocessing leakage occurs when preprocessing statistics (scaling mean/std, or here, the classification median threshold) are computed using validation or test data instead of training data alone. Test-set leakage occurs when the test set is used more than once during model development (repeated tuning, or accidentally training on test rows). This task specifically avoided: (1) computing scaling means/stds from anything other than `train_df`, (2) computing the classification threshold from anything other than `train_df`'s `CO(GT)` values, and (3) including `CO(GT)` itself, or the reference-analyzer columns `NOx(GT)/NO2(GT)` (correlated ground-truth pollutant readings from the same lab-grade equipment), as regression or classification features.

## 13. Limitations of Baseline ML

- Small effective dataset: after dropping rows with missing sensor readings, the usable dataset shrinks from ~9357 to ~7344 rows, and a single chronological split is used rather than multiple validation folds.
- Noisy/imperfect features: `NMHC(GT)` was excluded entirely due to being ~90% missing (marked -200), a known limitation of this dataset, not something introduced by this pipeline.
- Wrong or oversimplified targets: the `pollution_class` label is an arbitrary median split, not a domain-standard air-quality threshold (e.g. a WHO guideline value), so it may not reflect a practically meaningful boundary.
- Weak or linear-only features: linear/logistic regression cannot capture non-linear sensor response curves or interactions between features, which may genuinely exist in real sensor behavior.

- Class imbalance: while this task's median-split label is close to balanced by construction, a differently-defined threshold could easily produce a heavily imbalanced classification task, where accuracy alone would become misleading.
- Misleading metrics in isolation: R2, accuracy, or inertia alone can each look acceptable while hiding real problems (e.g. a model that ignores a minority class entirely, or a clustering with an inappropriate k) — multiple metrics and visual checks were used together in this task specifically to avoid this.

## 14. Theory Questions

### **1. What is the difference between regression, classification, and clustering?**

Regression predicts a continuous number (e.g. CO(GT)). Classification predicts a category from a fixed set (e.g. high/low pollution). Clustering groups data by similarity with no target or label involved at all.

### **2. What are features and targets?**

Features are the input columns used to make a prediction (e.g. sensor readings, temperature). The target is the value being predicted (e.g. CO(GT), or the derived pollution class).

### **3. What is the difference between supervised and unsupervised learning?**

Supervised learning trains on data where the correct answer (target) is known for every row (regression, classification). Unsupervised learning has no known answer — only features are available (clustering).

### **4. What is a baseline predictor?**

A deliberately simple predictor (e.g. always the training mean, or always the most frequent class) used as a minimum bar a real model must clearly beat to be considered useful.

### **5. Why is a model weak if it does not beat the baseline?**

If a trained model performs no better than a trivial rule that ignores the input entirely, it has not demonstrably learned any real, useful relationship between the features and the target.

### **6. Why do we need train, validation, and test sets?**

Train fits the model's parameters. Validation is used to tune choices (like learning rate) without touching test data. Test gives one final, honest estimate of performance on genuinely unseen data, since evaluating on training data overstates real performance.

### **7. Why should the test set not be used repeatedly during model selection?**

Repeated tuning against the same test set lets that set indirectly influence model choices over time, which defeats its purpose as an unseen check; the model effectively starts "fitting" to the test set indirectly.

### **8. What is gradient descent and what role does the learning rate play?**

Gradient descent iteratively adjusts model parameters in the direction that reduces loss, using the gradient (rate of change of loss with respect to each parameter). The learning rate controls how large each adjustment step is.

### **9. What happens if the learning rate is too high or too low?**

Too high: updates overshoot the minimum, and loss can oscillate or diverge instead of decreasing. Too low: updates are tiny, so training takes far more iterations to converge, or appears to stall before actually reaching a good minimum.

### **10. What is the difference between MAE, RMSE, and R-squared?**

MAE averages absolute errors (treats all error sizes proportionally). RMSE averages squared errors then square-roots back to original units (penalizes large errors more). R-squared expresses the fraction of the target's variance explained by the model, relative to a mean-only baseline.

**11. What is the difference between accuracy, precision, recall, and F1-score?**

Accuracy is overall fraction correct. Precision is how much of what was predicted positive was actually positive. Recall is how much of what was actually positive was correctly caught. F1 is a single balance of precision and recall.

**12. Why is accuracy misleading for imbalanced classification?**

If one class dominates, a model that always predicts that class can score high accuracy without learning anything real, since accuracy doesn't reveal how the minority class was handled.

**13. What does a confusion matrix show?**

The four possible outcome counts of a binary classifier's predictions vs actual labels: true positive, true negative, false positive, and false negative — revealing exactly what kinds of mistakes are being made, not just how many.

**14. What is inertia in KMeans?**

The sum of squared distances from every point to its own cluster's centroid, summed across all clusters — a measure of how tightly packed the clusters are.

**15. What is silhouette score?**

A per-point score (averaged across all points) measuring how much closer a point is to its own cluster than to the nearest other cluster, indicating how well-separated and well-matched the clustering is overall.

**16. What is data leakage? Give three examples relevant to this task.**

Data leakage is when information that would not be genuinely available at prediction time influences training, making performance look better than it really is. Examples in this task: (1) computing scaling mean/std from validation/test data instead of training data only, (2) computing the classification median threshold from anything beyond training data, (3) using CO(GT)-correlated reference-lab columns (NO<sub>x</sub>(GT), NO<sub>2</sub>(GT)) as regression/classification features.